

InkCalc: An Offline Embedded Handwriting Recognition Calculator

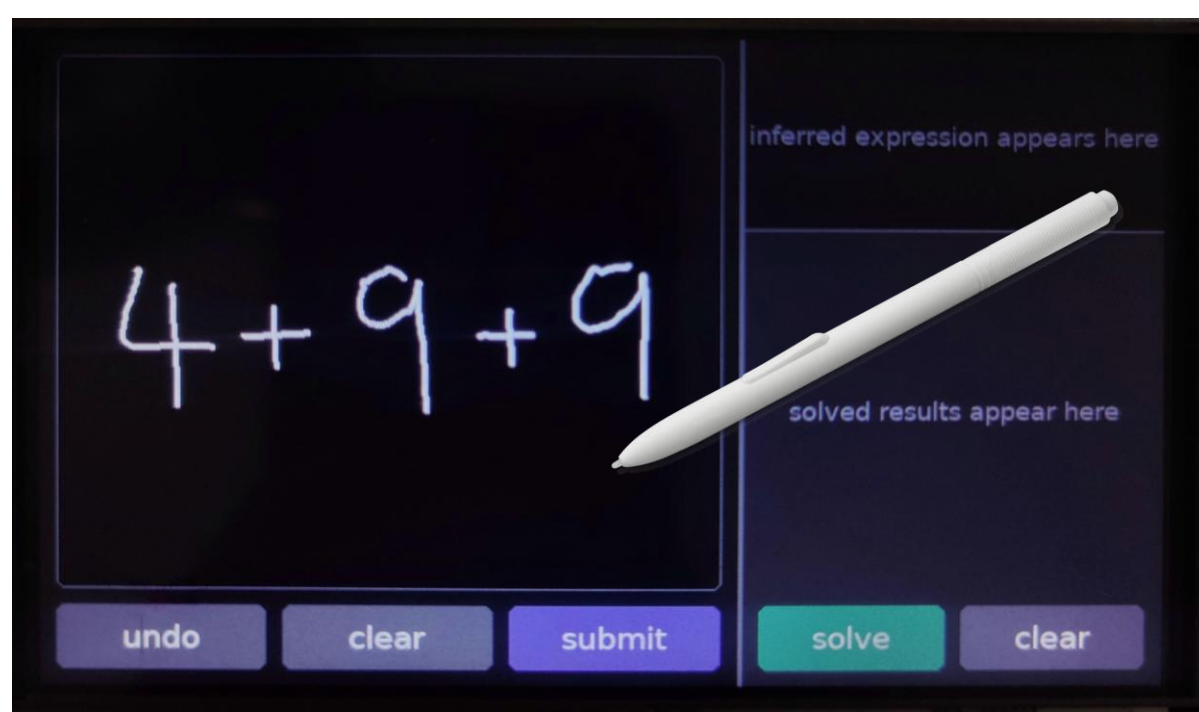


University
of Victoria

Alanna Cunha | Bowa Wang | Robin Zerr
Department of Electrical and Computer Engineering | ECE 499

INTRODUCTION

Writing mathematics by hand is intuitive, but transferring complex expressions to a calculator is often slow and requires knowledge of specialized input function formats. **InkCalc** is a handheld calculator that combines touchscreen handwriting input with on-device machine learning to provide an intuitive and efficient approach to solving mathematical expressions.

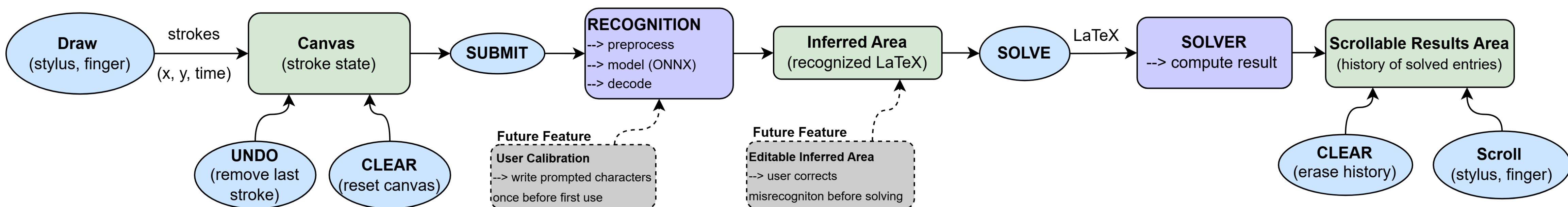


DESIGN AND FUNCTIONALITY

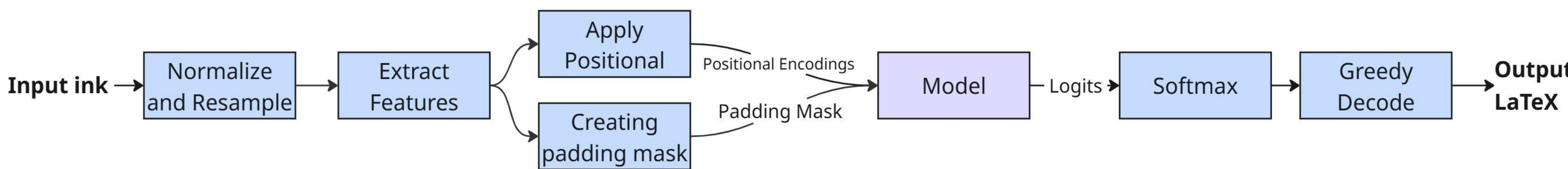
I. HARDWARE

- Raspberry Pi 4B + 5V/3A supply running the ONNX-exported model locally.
- 5" 800×480 resistive touchscreen (USB) with stylus input, capturing stroke data for the Draw step (Section II); same panel used as the HDMI display, functioning as both input surface and output screen.

II. HIGH LEVEL DEVICE FUNCTIONALITY: User Interaction and Workflow



III. DETAILED RECOGNITION PROCESS: Preprocessing and Postprocessing



Prior to training....

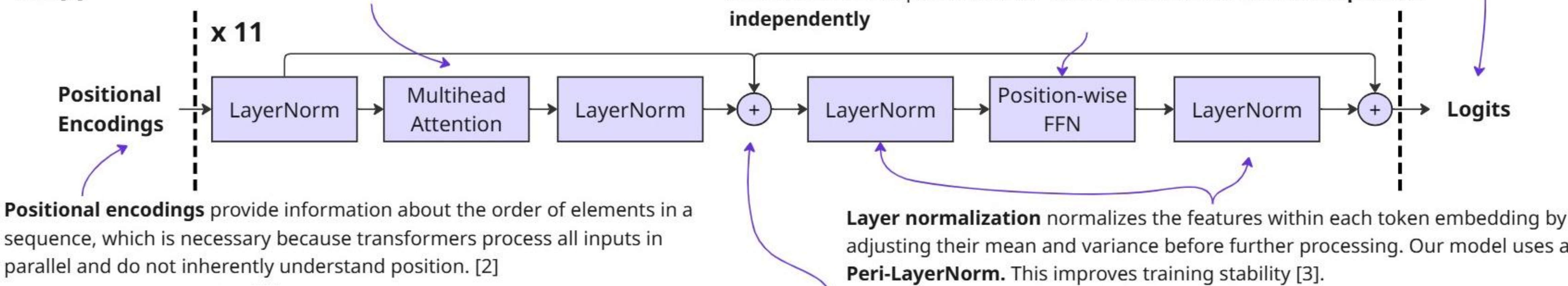
- Vocabulary is built by tokenizing ground-truth LaTeX labels into a fixed index set, which is used to encode targets for CTC training loss and decode predictions for evaluation scoring.

IV. MODEL DESIGN: CTC Transformer Architecture

Multi-head attention mixes information **across positions**, allowing tokens to "look at" one another. Each head focuses on different relationships within the data [2].

Logits are the raw output confidence scores generated by the transformer for each possible token at each time step.

Position-wise FFN performs a **non-linear transformation at each position independently**.



Positional encodings provide information about the order of elements in a sequence, which is necessary because transformers process all inputs in parallel and do not inherently understand position. [2]

Layer normalization normalizes the features within each token embedding by adjusting their mean and variance before further processing. Our model uses a **Peri-LayerNorm**. This improves training stability [3].

Residual addition adds the input of a transformer sublayer directly to its output through a skip connection.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

V. DESIGN ALTERNATIVES

- Selected a CTC Transformer over **Vision-Language Models** due to its significantly lower model size, direct support for InkML stroke data, and suitability for embedded deployment on a Raspberry Pi.
- Rejected RNN/LSTM architectures** in favor of the Transformer architecture for its superior handwriting recognition performance.

REFERENCES

- [1] P. Gervais, A. Fadeeva, and A. Maksal, "MathWriting: A dataset for handwritten mathematical expression recognition," *arXiv preprint arXiv:2404.10690*, 2024. ([arXiv](#))
- [2] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017. ([NeurIPS Papers](#))
- [3] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, "On layer normalization in the Transformer architecture," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119, pp. 10524–10533, 2020. ([Proceedings of Machine Learning Research](#))
- [4] H. Mouchère, CROHME: Competition on Recognition of Online Handwritten Mathematical Expressions, Univ. of Nantes / IRCCyN / LUNAM, Dataset Version 2.0, Oct. 2012. Available: http://www.iapr-tc11.org/dataset/CROHME/CROHME_full_v2.zip
- [5] J. Zhu, W. Zhao, Y. Li, X. Hu, and L. Gao, "TAMER: Tree-Aware Transformer for Handwritten Mathematical Expression Recognition," *arXiv.org*, 2024. <https://arxiv.org/abs/2408.08578>

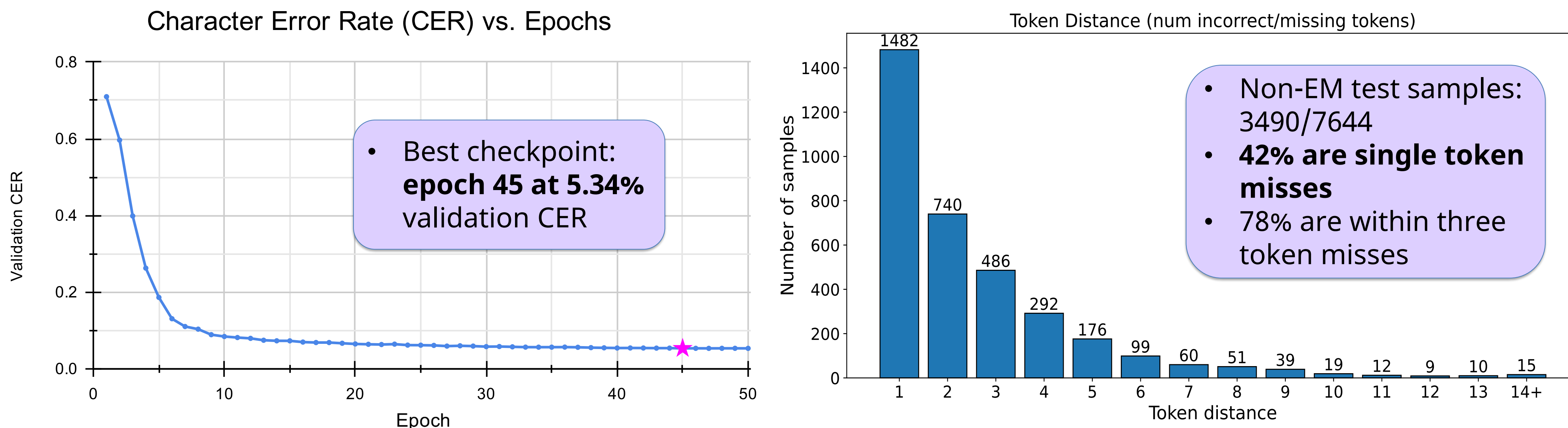
OBJECTIVES

- Develop a standalone embedded device for on-device handwriting recognition and mathematical computation.
- Implement a real-time pipeline with results returned within **5 seconds** of stroke completion.
- Design and optimize a lightweight **CTC Transformer** for handwritten mathematical expression recognition.
- Achieve **≤5.5% Character Error Rate (CER)** on the MathWriting test dataset [1].
- Develop an intuitive touchscreen interface for handwritten input and result visualization.

MODEL EVALUATION RESULTS

VI. RESULTS: Training, Validation, and Test

- Trained the model for 50 epochs using Adam optimizer with batch size 32 on 1×GPU w/ 8GB VRAM.
- Exact Match (EM) Rate of 54.3% (4154/7644) and Character Error Rate of 6.67% achieved on Test set.
- A reduction in CTC loss was found to not directly correlate to a reduction in CER.



VII. MODEL PREDICTION ERRORS

Common Error Cases on Test Dataset:

Error Category	Handwriting	Label (truth)	Model Prediction	What Went Wrong
Case Confusion		s)	S)	's' read as 'S'
Symbol Look-alikes		T	J	'T' read as 'J'
Structure and Grouping		$C_{\frac{1}{2}}$	$C_{\frac{1}{2}}$	dropped '{'

Catastrophic Case: Long, Vertically-Structured Expressions

Catastrophic		$\begin{bmatrix} 1 & 0 & -\vec{c}_D & \vec{z}_B \\ 0 & 1 & D & b \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & -\vec{c}_D & \vec{z}_B \\ 0 & 1 & D & b \end{bmatrix}$	20 tokens off (label is 36 tokens)
--------------	--	---	---	------------------------------------

CONCLUSIONS & FUTURE WORK

A prototype offline handwritten-input calculator was successfully developed, demonstrating end-to-end on-device mathematical recognition and evaluation. The system achieved **54.3% Exact Match**, **6.67% CER**, and **~3-second** average inference latency. With **42% of errors** consisting of single-token mistakes, the results demonstrate the feasibility of real-time, embedded handwritten mathematics recognition.

VIII. FUTURE WORK

- Expand training sets using MathWriting synthetic data and CROHME dataset [4] to improve generalization.
- Incorporate tree-aware mathematical representations to better preserve expression structure and reduce bracket/grouping recognition errors [5].
- Train the model on larger GPU resources with larger batch sizes to improve performance.
- Add support for transcendental functions and improve matrix recognition/evaluation capabilities.
- Enhance the user interface with user-specific handwriting calibration.
- Create custom casing and integrate keypad for expression editing.